

# Architect

How to use it

# Contents


| | | |
|---|---|---|
| 1 | What Architect does | 3 |
| 2 | Before you start | 4 |
| 3 | First run | 6 |
| 4 | Describe a job | 8 |
| 5 | Answer the teacher's questions | 11 |
| 6 | Check the job and the rule | 13 |
| 7 | Train | 17 |
| 8 | Results | 21 |
| 9 | Use your agent | 25 |
| 10 | Make it better | 28 |
| 11 | Share it | 32 |
| 12 | The command line | 36 |
| 13 | What leaves your computer | 39 |
| 14 | When something goes wrong | 41 |
| 15 | Glossary | 43 |


**How to read this guide**

Each chapter says what you will do, gives the steps, shows what you will see, and says what to do if something goes wrong. Words in **bold** are words on the screen. Commands and file names are in a typewriter font. The figures are the app's own screens.

# 1 What Architect does

You describe a job. A large model teaches a small one to do it, and the small one stays on your computer as your agent.

**Two models**

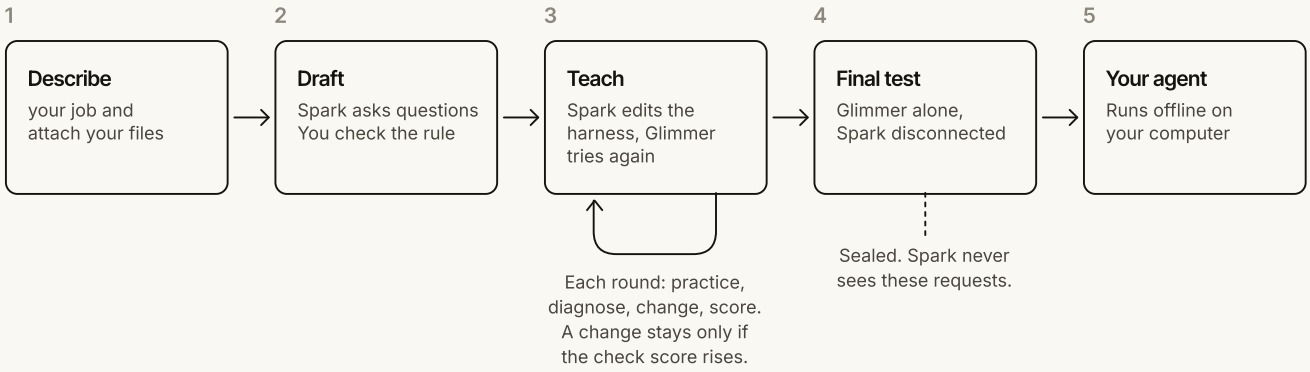
Architect uses two models from Meta. **Spark**, the teacher, is large. You reach it over the internet with a key. **Glimmer**, the student, is small. It runs on your computer through Ollama.

**What you do**

You describe a job and attach the files it works from. Spark reads them and asks about anything they leave open. Then it drafts the job. If your files include a policy with prices or rules, Spark turns it into a rule that you check and confirm.

**What Spark does**

Spark watches Glimmer try the job. When Glimmer gets something wrong, Spark finds the step where it went wrong and edits Glimmer's **harness**: its instructions, the notes it reads and small tools. A change stays only if Glimmer does better on check requests that Spark never sees. Spark sees the score and nothing else about them.




**The sealed test**

When training ends, Glimmer takes a final test on its own, with Spark disconnected. The requests in it were put aside before training started. Spark never sees them. If Glimmer passes, it graduates and becomes your agent.

**Your agent**

The agent runs offline on your computer. It works on your files and asks before it changes one. It talks to nothing except the model on your computer. Later you can teach it more, keep versions, share it, and run it with the `architect` command.

# 2 Before you start

Check four things first: your computer, Ollama, a key for the teacher, and what teaching will cost.

### Your computer

Architect runs on Windows and macOS. Glimmer runs on the graphics card, so the size of its memory decides which build of Glimmer you can use.

On Windows, Architect reads the memory of an NVIDIA card. On a Mac with Apple silicon, the graphics chip shares the computer's memory, and Architect counts about 70 percent of it. On other hardware it can't tell, and it recommends the larger build.

| Build | Download | On disk | Graphics memory |
|---|---|---|---|
| Glimmer, recommended. Most accurate. | 17 GB | 17 GB | 20 GB or more |
| Glimmer, smaller. Lighter, but less accurate. | 14 GB | 10 GB | 11 GB or more. Made for a 12 GB card. |

If a build doesn't fit, part of it runs on the processor. It still works, but it is slower. The first setup screen says so: **Part runs on the processor, so it is slower**.

### Ollama

Glimmer runs through Ollama, a free app from ollama.com. Install it and open it before you start. Architect checks that it is running.

### A key for the teacher

Spark needs a key from Meta. The setup screen has a button for it, **Get one from Meta**.

You can choose another teacher instead. Sign in to OpenRouter and pick any of its models. Use a key from OpenAI, Anthropic, Gemini or another service. Or use a model on your own computer, in Ollama or LM Studio. A teacher on your computer is free, and nothing you send leaves your computer.

Keys are stored encrypted on your computer.

### Two optional accounts

An OpenRouter account lets Glimmer practice on an online copy of itself, which is about ten times faster. A Hugging Face token is needed to publish an agent. It is also needed if you let Spark retrain Glimmer's weights.

## What teaching costs

| Part | Cost |
|---|---|
| Glimmer on your computer: practice, the final test, chats | Free |
| Spark, the teacher | Billed by its provider for the text it reads and writes. Meta lists Spark 1.3 at $1.25 per million tokens read and $4.25 per million written. |
| Online practice, if you choose it | Billed by the online host. The Define screen shows an estimate. |

Before training starts, the Define screen shows an estimated time and cost, and a spending cap for Spark. A job you describe starts with a $5 cap, and you can change it. Training stops before it goes over. In a returns desk run the Architect team measured, teaching cost 42 cents of Spark.

Settings, under Training, has **How hard Spark thinks**. More thinking can find subtler fixes but costs more per round.

Time matters more than money. On your computer Glimmer takes about 2.5 minutes a task, so rounds are slow and the final test of 60 tasks takes about 2.5 hours. A job starts with a time limit of 4 hours. If not even one round fits, Architect says so and offers more time or online practice (chapters 6 and 7).

# 3 First run

Four screens set Architect up. They take a few minutes. The model downloads while you carry on.

### Opening it

The app isn't signed yet, so your system may warn you the first time. On Windows, choose **More info**, then **Run anyway**. On a Mac, right-click the app and choose **Open**.

### Steps

The steps are named at the top of the window: **Model**, **Teacher**, **Faster training** and **Done**.

1. **Model.** Architect picks the build of Glimmer that suits your graphics card and marks it **Recommended**. Press **Download Glimmer**. The download is large and continues while you finish setup. You can choose the smaller build, press **Download later**, or open **Use a different model** to pick a model that is already on your computer. See **Figure 1**.
2. **Teacher.** Paste your Meta key into **Meta API key** and press **Test connection**. A message says **Connected** when it works. **Other ways to connect a teacher** shows the other choices: **OpenRouter**, **Another service** and **On this computer**. See **Figure 2**.
3. **Faster training.** Optional. Sign in to OpenRouter, or use a key, so Glimmer can practice on an online copy of itself. The finished agent still runs only on your computer. Press **Skip** if you don't want it.
4. **Done.** The last screen says **You're set up** and shows the state of Glimmer and Spark. Press **Get started**.

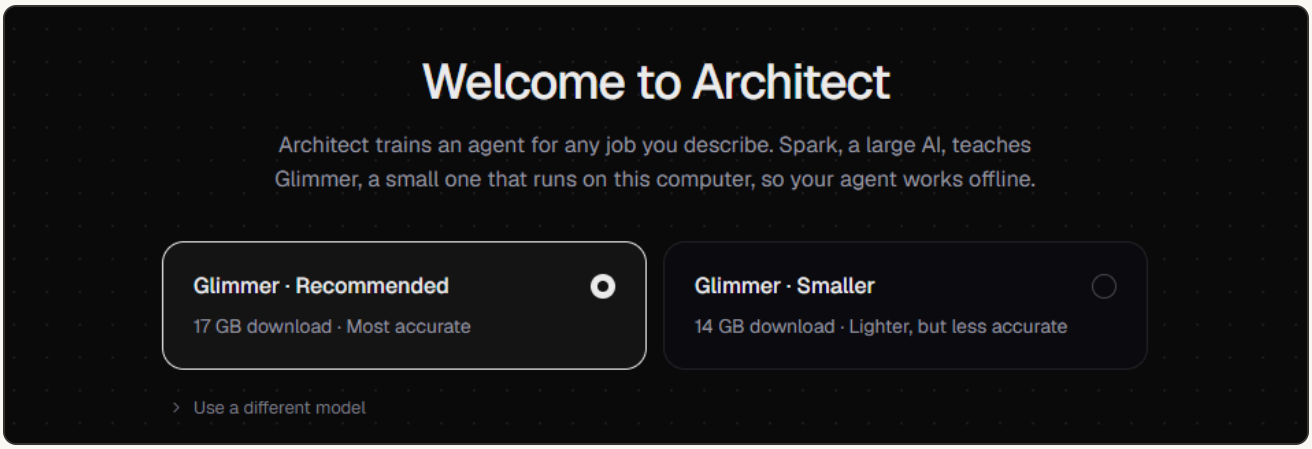



Figure 1

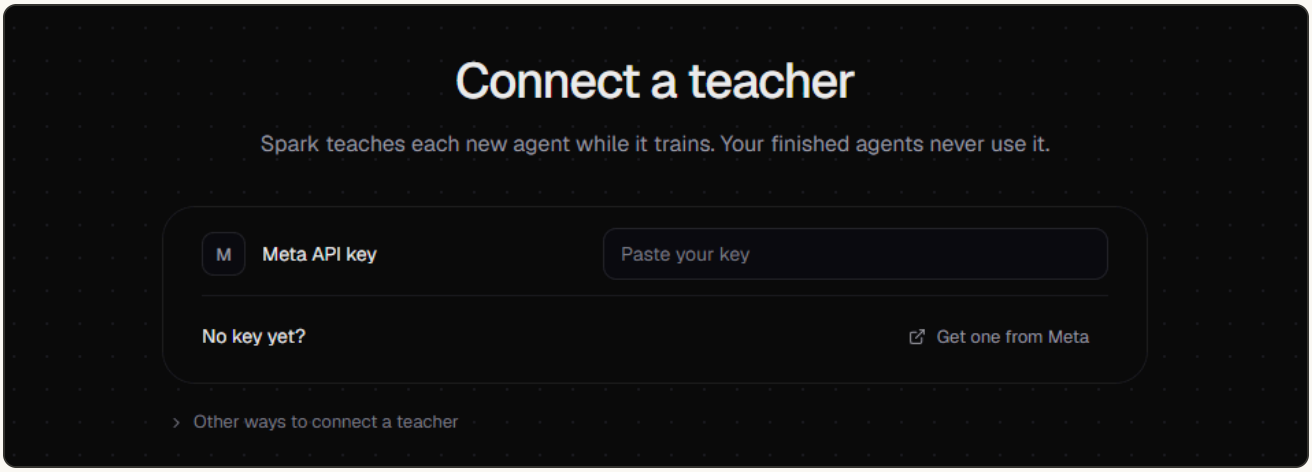



Figure 2

### What you will see

The home screen. With no agents yet it says **No agents yet** and offers three buttons: **New agent**, **Try the sample job** and **Try a ready-made agent**. The sample job is a small job that Architect trains an agent for while you watch. The ready-made agent is already trained, so you can start using it at once.

### If something goes wrong

| If you see | Do this |
|---|---|
| **Ollama isn't running** | Open Ollama. If you don't have it, press **Get Ollama**. Then press **Check again**. |
| **The download fails** | The screen shows the error. Check that you have enough free disk space, then press **Try again**. |
| **The key test fails** | The message says why. Check that you pasted the whole key and that it is a Meta key. You can add or replace keys later in Settings, under Models. |
| **You pressed Skip for now on the teacher** | Architect can't train a new agent until a teacher is connected. Connect one in Settings, under Models. |

# 4 Describe a job

Tell Architect what the agent should do, and attach the files it works from.

**Steps**

1. Press **New agent** in the sidebar. The screen asks **What should your agent do?**
2. Write the job in your own words in the box.
3. Press the paperclip and pick the files the job works from. You can pick several at once.
4. Look at the corner of the box. **Teacher: Spark** names who will read what you send. A cloud means it is online. A laptop means it runs on your computer.
5. Press the arrow to send. See **Figure 3**.

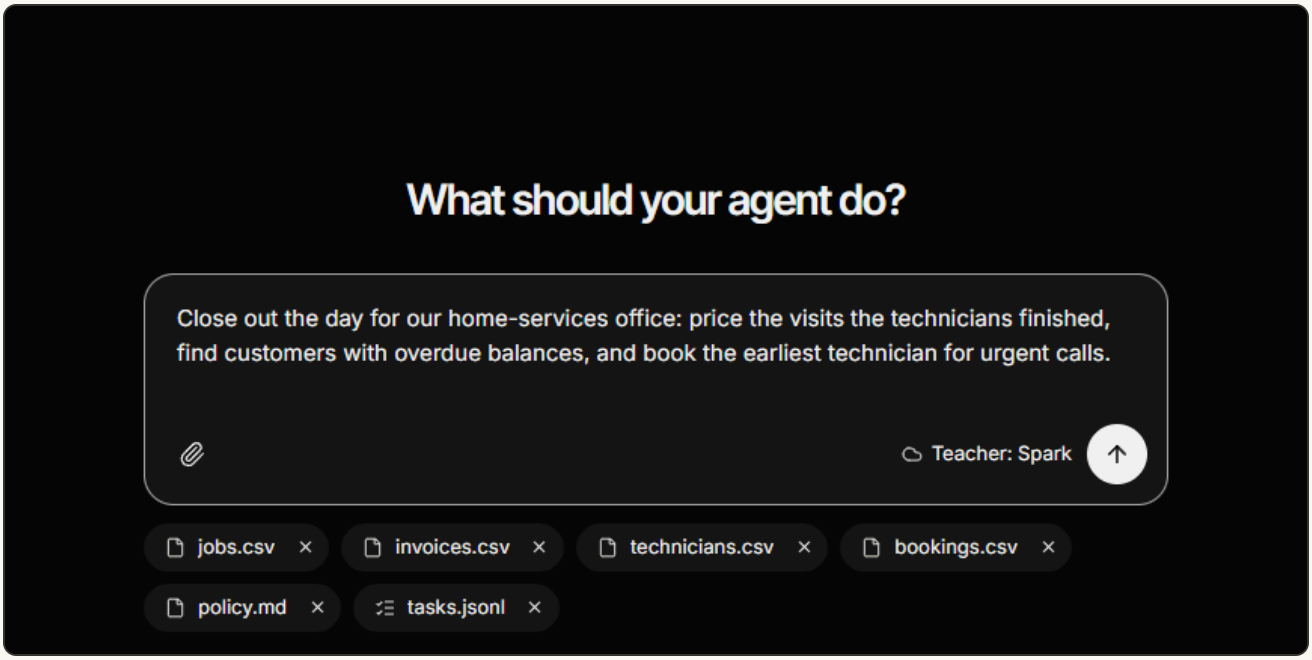



Figure 3

## Files that work

| Type | Use it for |
|---|---|
| `.csv` | Records the agent looks up and works out, such as invoices, jobs or tickets. Architect makes practice tasks from the rows. |
| `.txt`, `.md` | A policy, a rate card or rules. A text file is read whole, up to 30,000 characters, and 60,000 across all text files. Attach just the part the job needs. |
| `.jsonl` | Optional. Example tasks with their answers, one task per line. Architect uses them as they are. |

You need at least 30 rows of records in all, or 30 example tasks. You can attach up to 12 files, 25 MB each and 60 MB together.

Not yet supported: spreadsheets such as `.xlsx`, PDFs, Word documents and JSON files. Save or export them as CSV or text first.

## Writing the description

- Say what the agent should do, as tasks. "Price the visits the technicians finished" is better than "help with billing".
- Say what each file holds if its name doesn't.
- Say what it must not do. Spark lists this under **What it won't do**.
- Say how good it has to be if you have a target. If you don't, Spark sets one and you can change it (chapter 6).
- Put prices and rules in a file, not only in the description. Architect turns the file into a rule you can check.

### A good description

**Close out the day for our home-services office: price the visits the technicians finished, find customers with overdue balances, and book the earliest technician for urgent calls.**

It names three tasks. The files fit them: `jobs.csv`, `invoices.csv`, `technicians.csv`, `bookings.csv`, `policy.md` and `tasks.jsonl`.

### A weak description

**Help me with my business.**

It names no task and no limits, and it has no files. Spark has nothing to make practice tasks from, and nothing says what a right answer looks like.

## Other ways to start

Under the box you will find **Try the sample job**, **Try a ready-made agent** and **Import a job file**. Importing starts from a job someone made, with its tasks and data beside it. Choose its `job.json`.

### What you will see

Spark reads your description and files. The screen says so, then shows **Read 6 files** or however many you sent. If it takes a while, it says **Still working. This can take a minute.** Spark then asks a question (chapter 5) or says it has drafted the job.

### If something goes wrong

| If you see | Do this |
|---|---|
| **Your files hold only a few rows** | Attach more of the records, or a `tasks.jsonl` file with at least 30 example tasks. Then describe the job again. |
| **A file is "a spreadsheet", "a PDF" or "a Word document"** | Save or export it as CSV or text, and attach that. |
| **A text file is longer than 30,000 characters** | Attach only the section the job needs, such as the part on pricing. Architect won't read part of a file, because a rule read from part of a policy can miss the exception that matters. |
| **Nothing to make practice tasks from** | Attach the records as CSV files, or a `tasks.jsonl` file, and describe the job again. |
| **The draft couldn't be used** | Try again, or describe the job a little differently. |
| **Spark couldn't answer** | Press **Try again**. If it keeps failing, check the teacher's key in Settings, under Models, or press **Edit the description**. |

5

# Answer the teacher's questions

Spark asks only what your files leave open. Your answers change what the agent does and how its answers are checked.

## Steps

1. Read the question. Spark asks one at a time and offers two to four short answers as buttons.
2. Press the answer that is true for your business. If none fits, type your own in the **Reply** box.
3. Spark asks at most three questions. At any point you can choose **Draft it now** to make it use what it has.
4. When it has enough, it says **That's everything I need. I've drafted the job for you to review.** Press **Review the job**. See **Figure 4**.

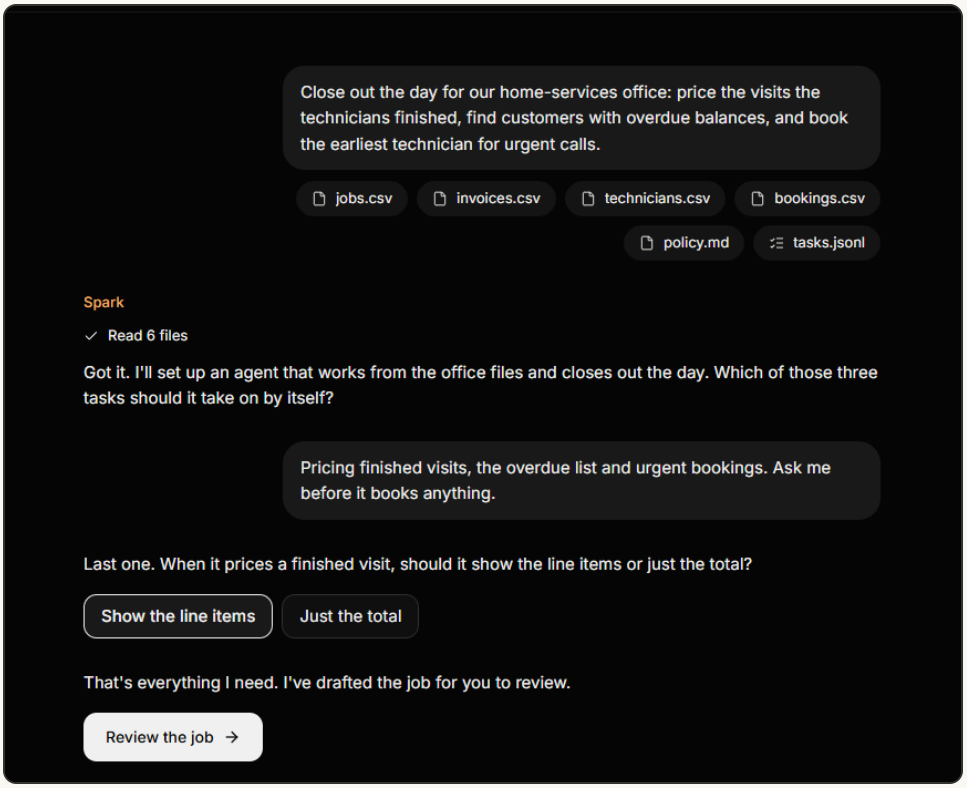



Figure 4

### What you will see

Your message and files appear first, then Spark's reply. Some questions come from Architect itself. If your records have empty cells, or marks such as N/A or a dash where a number belongs, Architect asks what to do with them: **Count it as zero** or **Leave those rows out**.

Those answers change the numbers the agent is tested on, so they need a real answer. **Draft it now** does not stand in for one.

### Answering well

- Answer for what happens in your business, not for what sounds right. If you are unsure, pick the safer option.
- Short answers are fine.
- If you notice your description was wrong, press **Edit the description**. Your words and files come back as you left them.

### If something goes wrong

| If you see | Do this |
|---|---|
| **Spark couldn't answer** | Press **Try again**. |
| **This conversation is gone** | Architect forgets a conversation left alone for about an hour. Press **Edit the description** and send it again. |
| **The conversation ends with no job** | The message says what is missing. Press **Edit the description**, add what it names, and send again. |

# 6 Check the job and the rule

The Define screen shows the job Spark drafted. Read it, check the rule if there is one, choose where to train, and start.

### What is on the screen

Top to bottom: the job's name and summary, with a note that once trained it runs offline on your computer. **What it does** and **What it won't do**. **Check the rule**, if your files include a policy. **Where should it train?** with its estimates and a spending cap. **Ready when** and **Permissions**. At the bottom, **Advanced settings**, **Back** and **Start training**. See **Figure 5** and **Figure 7**.

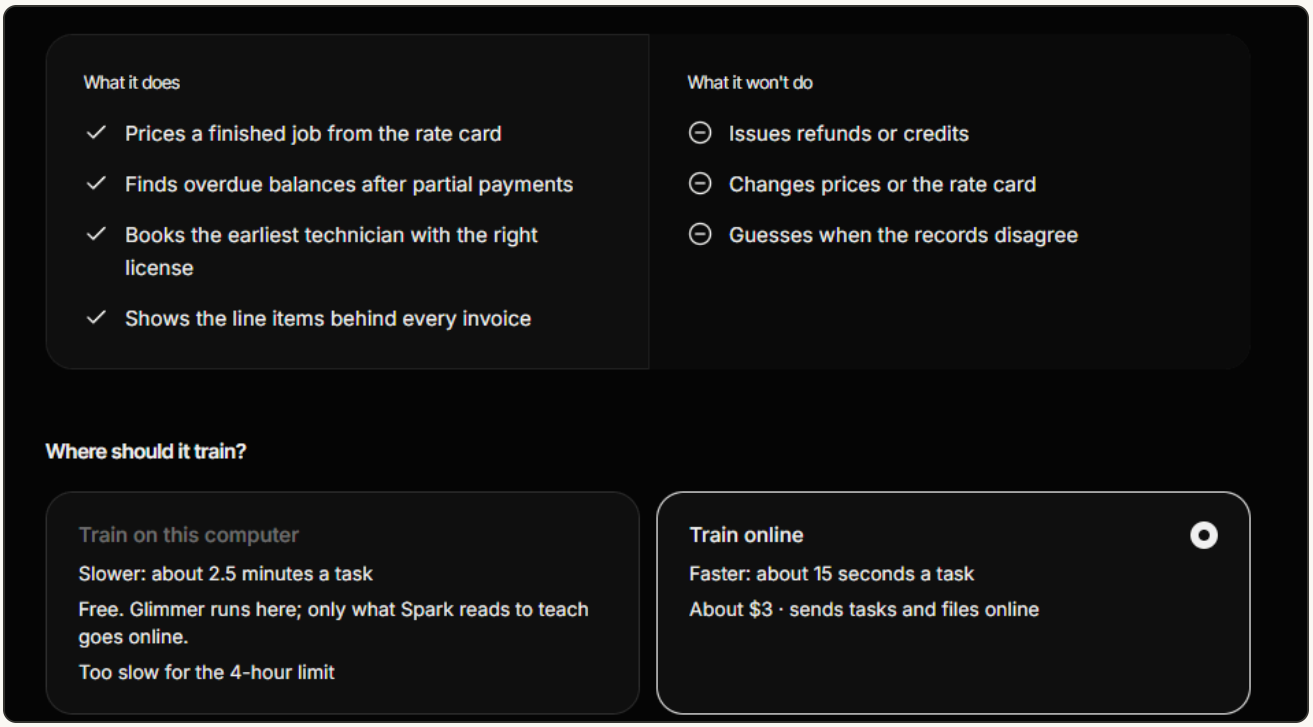



Figure 5

### Steps

1. Read **What it does**. Each line should be something you want. If one is wrong, press **Back** and describe the job again.
2. Read **What it won't do**. These are limits the agent keeps.
3. If the screen has **Check the rule**, do that next (below).
4. Choose where to train (below).

5 Look at **Ready when**. It is the share of new tasks the agent must get right, and training stops early once the check tasks reach it. To change it, open **Advanced settings**, then **Ready when**.

6 Look at **Saving files**. **Ask first** shows you each change to a file and waits. **Allow** saves without asking. **Never** means it can't change your files. You can change this again later.

7 Press **Start training**.

### Check the rule

If your files include a policy with prices, discounts or rounding, Spark turns it into a rule. Architect works out the right answer to every task from that rule and grades Glimmer against those answers. If the rule is wrong, the grades are wrong too. So nothing trains until you confirm it. See **Figure 6**.

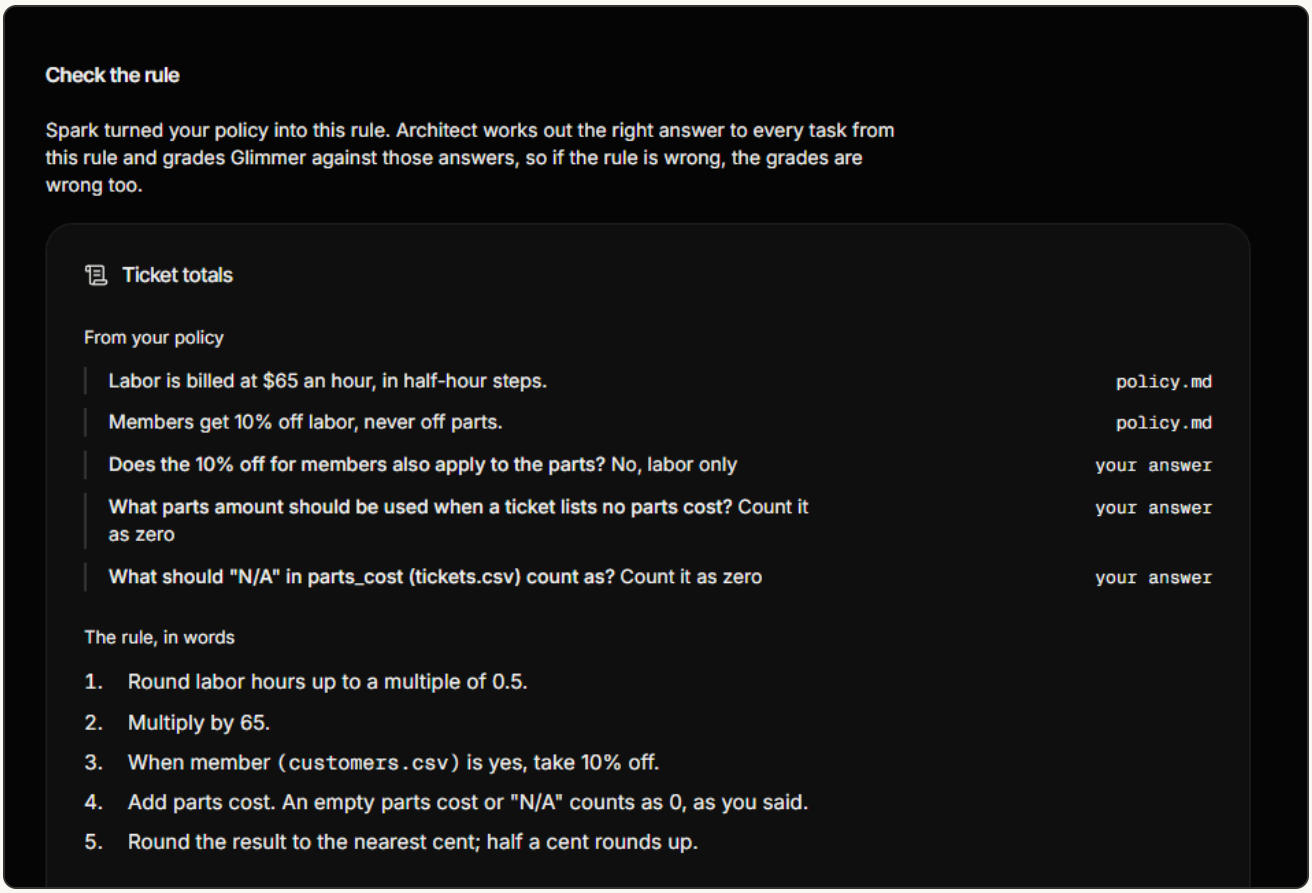



Figure 6

1 Under **From your policy**, read the lines Spark took. Each shows where it came from: a file, or **your answer**. Are they the right lines?

2 Under **The rule, in words**, read the numbered steps. Is that how your business works?

3. Under **Examples**, pick one and work it out yourself. Every example shows each step. An example tagged **Not from your records** was made up to show how the rule rounds.
4. Read the notes at the bottom. They say how many rows were left out of the tasks, and why.
5. If all is right, press **Confirm the rule**. If not, press **Change the description**. Your words and files come back as you left them, and Spark may ask its questions again.

> **Start training** stays off, and the bar says **Confirm the rule first**, until you confirm. If your records change afterwards, a badge says **Changed since you confirmed it**. Press **Describe the job again** so the tasks are made from the records as they are now.

The Results screen later repeats the point: the score shows how often the agent agreed with the rule you confirmed. It can't show whether the rule is right for your business. Only you can check that.

### Where should it train?

| Choice | What it means |
|---|---|
| **Train on this computer** | Free. Glimmer runs here, and only what Spark reads to teach goes online. Slower: about 2.5 minutes a task. |
| **Train online** | Glimmer practices on an online copy, about 15 seconds a task. The host bills your account, and tasks and files are sent online. |

**Estimated time** and **Estimated cost** add up the training, the final test and the bills. **How this is estimated** explains them. The **Spark spending cap** slider sets the most Spark may cost, and training stops before it goes over.

The final test runs on your computer unless you choose the online copy in **Advanced settings**, under **Ready when**. An agent tested online is marked **Passed online** until you test it on your computer too (chapter 8).

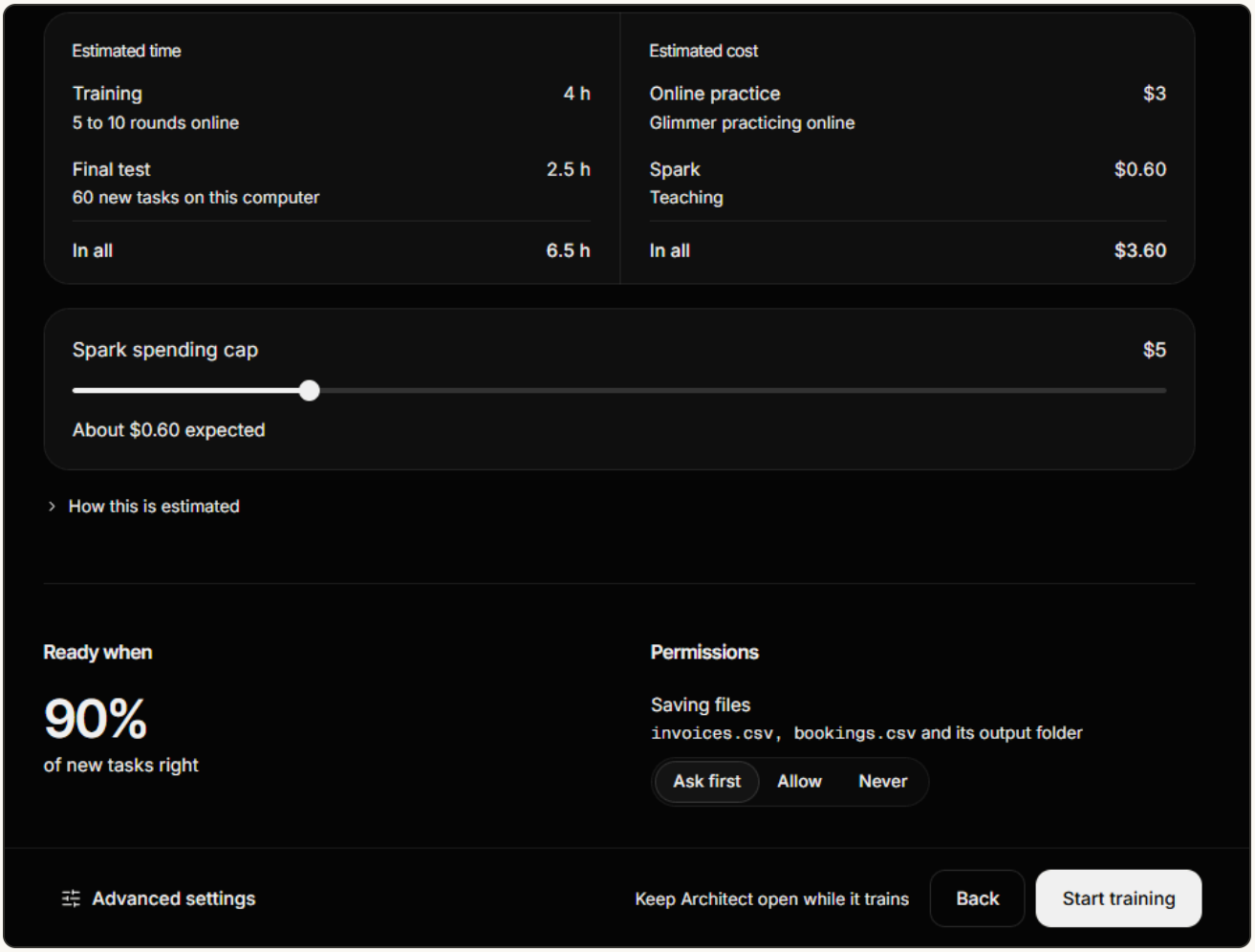



Figure 7

## If something goes wrong

| If you see | Do this |
|---|---|
| **Not even one round of teaching fits in the time** | Scoring the check tasks and one round take longer than the limit. Press the button that offers more time, or **Train online instead**. |
| **A line saying something is missing, such as Ollama isn't running** | Each line has a button that puts it right, such as **Check again**, **Download** or **Add its key**. **Start training** waits until all are done. |
| **Online training isn't set up yet** | Press **Set up online training** and add the service's key. |
| **Spark can't change anything yet** | Every kind of change that could run here is switched off. Open Spark's tab in **Advanced settings** and switch one on. |
| **Fix the marked fields in Advanced settings first** | A setting is out of range. Open **Advanced settings** and correct the fields it marks. |

# 7 Train

Spark teaches Glimmer in rounds. You can watch, answer a question, pause or stop. Every limit is a hard cap.

## Steps

1. Press **Start training**. Keep Architect open while it trains.
2. Watch the Train screen. The large panel shows Spark and Glimmer and what each is doing. See **Figure 8**.
3. If Spark has a question, answer it in the round panel (below).
4. When training ends, Glimmer takes the final test by itself. Then press **See the results**. If you are at the top of the page, the app opens the results for you.

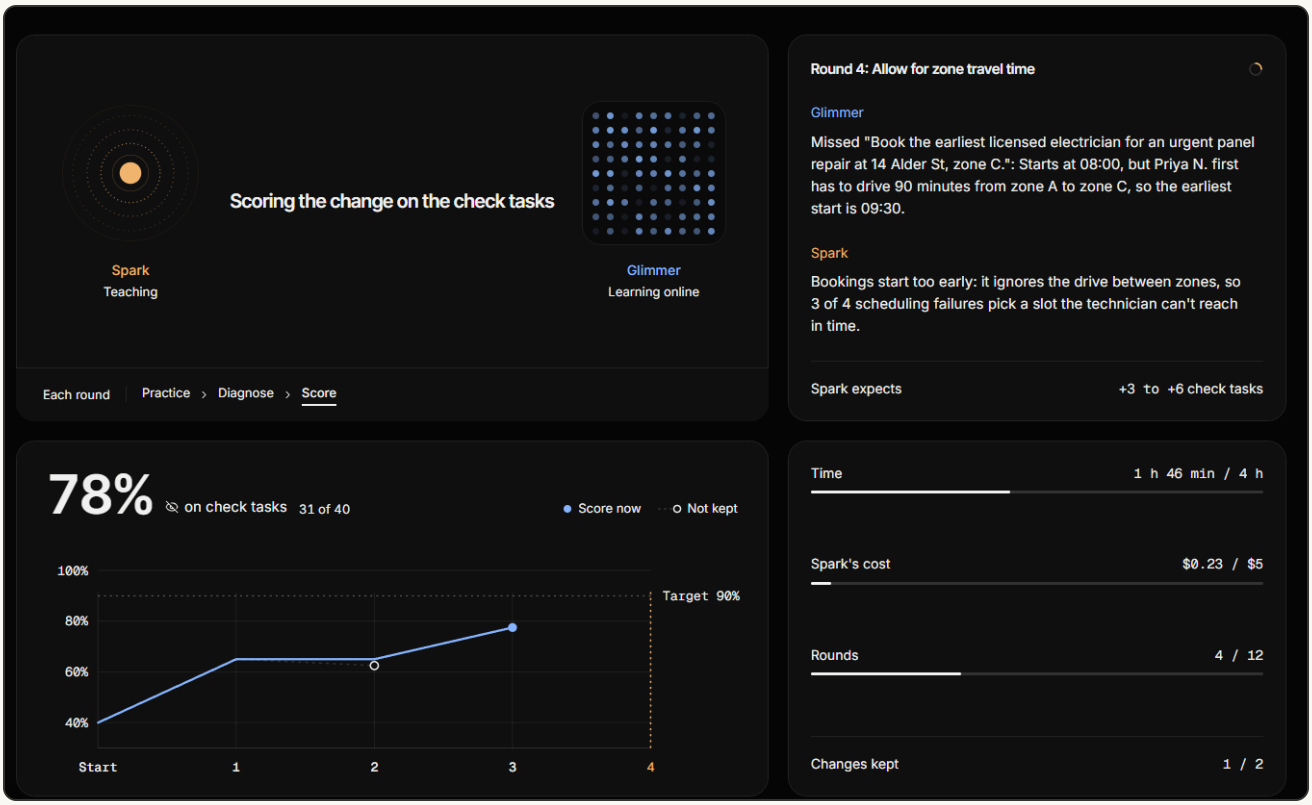



Figure 8

## What a round is

Before round 1, Architect scores Glimmer on the check tasks, so there is a starting score. If the job starts blank, round 1 is a first draft: Spark writes a whole set of instructions from the job and the example tasks. After that, each round has four steps.

1. **Practice.** Glimmer works through a set of practice tasks.
2. **Diagnose.** Spark reads the failures and finds the step where Glimmer went wrong. It picks one change and says what it expects, for example **+3 to +6 check tasks**.
3. **Change.** Architect applies the change to a copy of the harness. Spark can change the instructions and steps, what Glimmer reads and when, and small tools. It can also ask you a question. Retraining Glimmer's weights can't run without a Hugging Face key and a training budget.
4. **Score.** Glimmer takes the check tasks again with the change.

The check tasks are separate from the practice tasks. Spark sees only their score.

### Kept or undone

The **Rounds** list shows how each change ended.

| Outcome | What it means |
|---|---|
| **Kept** | It gained enough on the check tasks and broke no rule. The agent keeps it. |
| **No gain** | It didn't gain enough, so it was undone. |
| **Made it worse** | It made the check tasks worse or broke a rule, so it was undone. |
| **Couldn't be tried** | It couldn't be tried in this run, so nothing changed. |
| **Stopped** | Training ended before it was scored. The agent never took it. |

For a job you describe, a change must gain at least 2 check tasks, and no task type may lose more than 1. A promising change is measured twice, to rule out luck. You can change these numbers in **Advanced settings**, under **When a change stays**. See **Figure 9**.

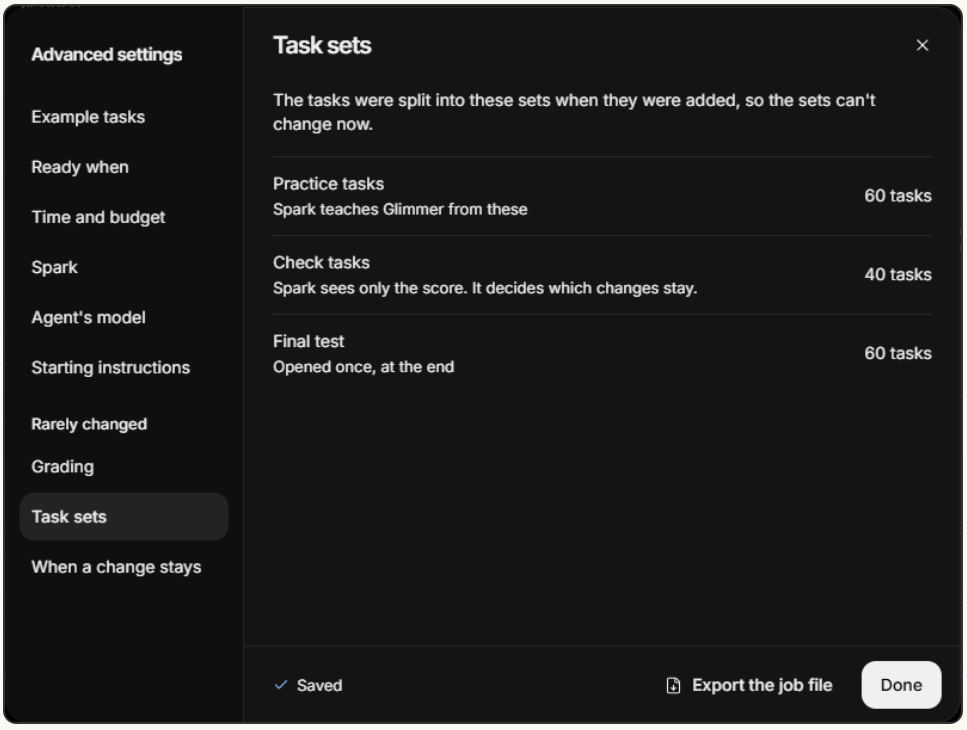



Figure 9

### The rest of the screen

- The score chart shows the share of check tasks right after each round, with the target as a dotted line.
- **Time**, **Spark's cost** and **Rounds** show how much of each limit is used. **Changes kept** counts the changes kept out of those tried.
- **Worked examples** lists corrected examples, each marked **Checker passed it** or **Checker failed it**. It says **No worked examples yet** until there are some.
- **Rounds** has a row for each change. Open a row to see the diagnosis and the scores before and after.
- **Latest attempts** lists Glimmer's attempts. Open one to step through what it did.

### When Spark asks you

Sometimes Spark stops to ask one question. The run pauses, the sidebar row says **Needs you**, and the round panel shows the question. Type your answer in **Your answer** and press **Send answer**. Spark keeps it as what you meant and sees it in every round after.

Tick **This changes what counts as success** only if your answer really does. The run then stops, because the final test belongs to the job as it was. You would train again on the changed job.

### Limits

Rounds, Spark's dollars and time are hard caps. Architect checks them before every call to Spark and before every round. A call that would go over a cap does not start. A job you describe starts with 8 rounds, $5 and 4 hours. Paused time doesn't count. Practice still running when the time is up is cut off and not scored, and the run goes on to its final test.

Training ends when it reaches the target, uses a limit, or stops finding gains. The Rounds list closes with the reason, such as **Reached the target**, **Used all 8 rounds**, **Spark's budget ran out**, **The time limit ran out** or **No gain in 4 rounds in a row**.

### Pause and stop

**Pause** holds training. **Resume** carries on from where it stopped. **Stop** ends training after the step in progress. If a change has been kept, the box asks **Stop training?** and offers **Stop and run the final test**: the agent keeps its best version so far and Glimmer takes the final test without Spark. If no change has been kept, a stop makes no agent.

You can leave the screen. Training goes on, and the sidebar lists the job as **Training**, **Paused**, **Needs you** or **Results ready**. Architect saves every step. If it closes, open it again and find the job in the sidebar.

### If something goes wrong

| If you see | Do this |
|---|---|
| **Training is paused, with a list** | A run pauses when something it needs isn't there: a key (**Add the key**), Ollama (**Check again**) or the model (**Download**). Put it right, then press **Resume**. |
| **Training stopped on an error** | The message says why. Press **Back to the job** and start again. |
| **Training didn't start** | Spark had nothing it may change. The notice says what is off. Press the button on it to turn a change back on. |
| **Spark's budget ran out before the target** | The run ends and Glimmer takes the final test. Raise the cap in **Advanced settings** and train again, or improve the agent later (chapter 10). |
| **Rounds end in No gain** | Open those rounds to see what Spark tried. Add more of your records or example tasks, and describe the job again. |
| **The computer is loud or hot** | Turn on **Quiet local models** in Settings, under Training. It uses fewer processor cores and takes short rests, so it is slower but quieter. |

# 8 Results

The Results screen says whether Glimmer graduated, how it did on a test it had never seen, and what it cost.

**Steps**

1. Read the pill and the title at the top, such as **Ready** and **Service desk graduated**. The table below lists the pills.
2. Read the sentence under the title. It says how many tasks it was tested on, where, and that Spark was disconnected.
3. Look at the four tiles: score, answer time, model and cost. See **Figure 10**.
4. Look at **By task type** to see which kinds of request it does well and which it doesn't. See **Figure 11**.
5. Press **Start using** and the agent's name to accept it and open a chat. Press **See all rounds** to look back at training.

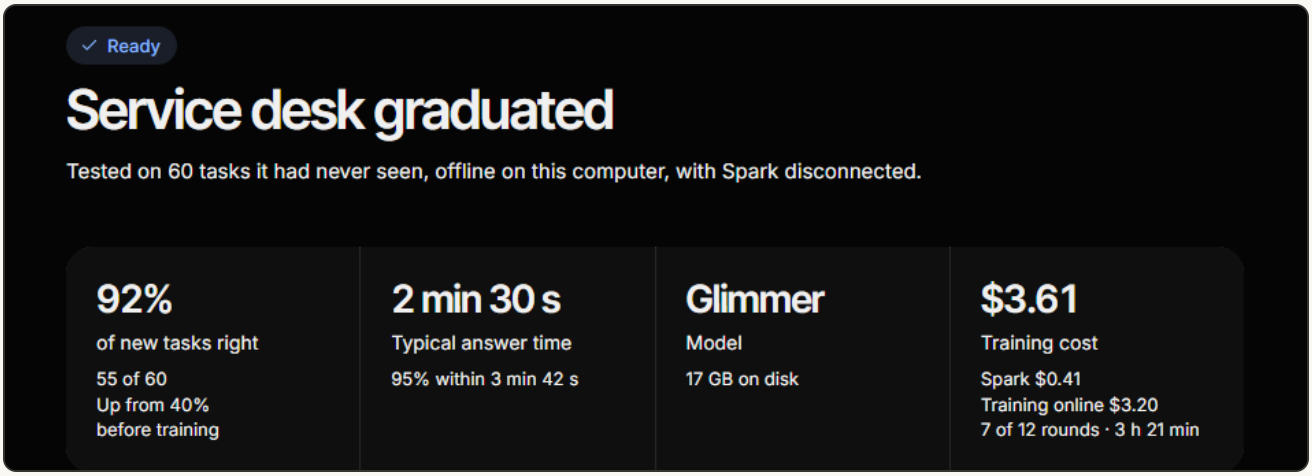



Figure 10

## The pill

| Pill | What it means |
|---|---|
| **Ready** | It passed its final test on your computer, and this computer's checks passed. |
| **Passed online** | The final test ran on the online copy. Test it on your computer to mark it ready. |
| **Not ready** | It missed the target. The screen says by how much. You can still use it. |
| **Needs attention** | One of this computer's checks needs something from you. |
| **Can't run** | A check on this computer failed. The button reads **See what's wrong**. |

## What the numbers mean

| Tile | What it shows |
|---|---|
| **of new tasks right** | The share of final test tasks answered correctly, such as 55 of 60. A line such as **Up from 40% before training** compares it with Glimmer's score on the same test with no teaching, when that was measured. |
| **Typical answer time** | The middle time for a task. **95% within** gives the time all but the slowest 5% took. |
| **Model** | Which model, and its size on disk. |
| **Training cost** | Spark's bill and any online practice, with the rounds and time used. |

Three sets of tasks are kept apart, and the app tells them apart in every table.

| Set | What it is for |
|---|---|
| **Practice tasks** | Spark teaches from these, and sees Glimmer's attempts at them. |
| **Check tasks** | Spark sees only the score. The score decides which changes stay. |
| **Final test** | Opened once, at the end. Spark never sees it, and nothing about it goes back into training. |

> Trust the final test. The check score helped choose the changes, so it can look better than the agent really is.

If you attached the tasks, the final test is part of them. Otherwise Architect made its requests from your files, following the job Spark drafted. The answers are checked against that job. Nobody has certified them as right for your business.

### The rest of the page

**By task type** gives the final test result for each kind of request. **What it won't do** lists what the agent leaves to you. **Comparisons** sets this agent beside other setups on the same tasks: Glimmer untouched, Glimmer with Spark's setup and, where it was measured, Spark doing the tasks itself. A row says **Not run** where nothing was measured. Architect never fills in a number it didn't measure. **What helped most** lists the rounds that gained the most check tasks.

If you switch on **Comparison mode** in Settings, under Training, Architect also trains three simpler versions of each agent: Glimmer on its own, a setup Spark writes in one go, and Glimmer revising its own. They appear under **Compared with simpler methods**.

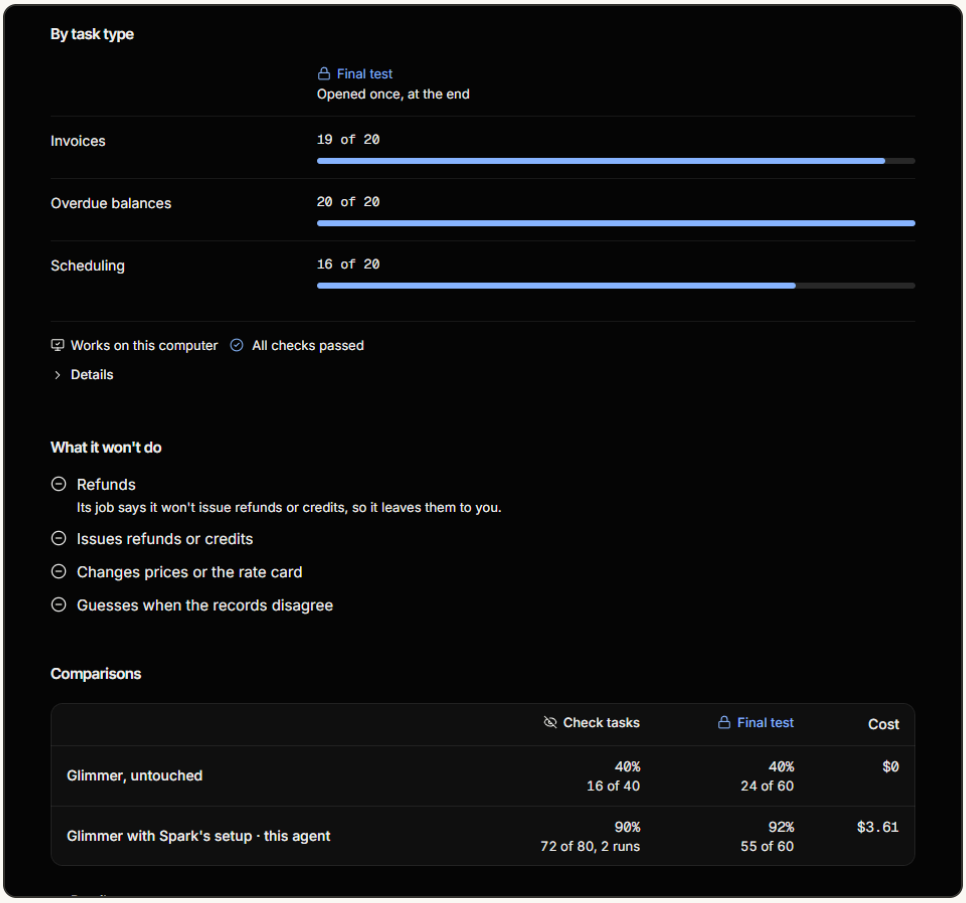



Figure 11

### Testing on this computer

If the final test ran on the online copy, the screen offers **Test on this computer**. It takes about 2.5 hours for 60 tasks and uses the graphics card. You can leave the page while it runs.

### If something goes wrong

| If you see | Do this |
|---|---|
| **Not ready** | The agent can still be used, but it missed its target. Teach it more (chapter 10), or train again with more files. |
| **Can't run on this computer** | Press **See what's wrong**. The agent's General tab lists this computer's checks and has a button for each fix. |
| **Couldn't load the results** | Press **Try again**. |

# 9 Use your agent

Chat with your agent, let it work on your files, and approve each change. It works offline.

## Steps

1. Press **Start using** on the Results screen. A chat opens. Next time, pick the agent in the sidebar.
2. Ask in plain words in the box that says **Ask** and the agent's name. A new chat offers a few requests to start with, taken from what the agent passed on its test.
3. Watch the reply. The line above it, such as **Worked for 2 min 21 s**, opens to show each step: what the agent read, searched or ran. See **Figure 12**.
4. If it wants to change a file, it shows the change first (below).
5. The plus beside the agent's name starts a new chat. Each chat keeps its own history.

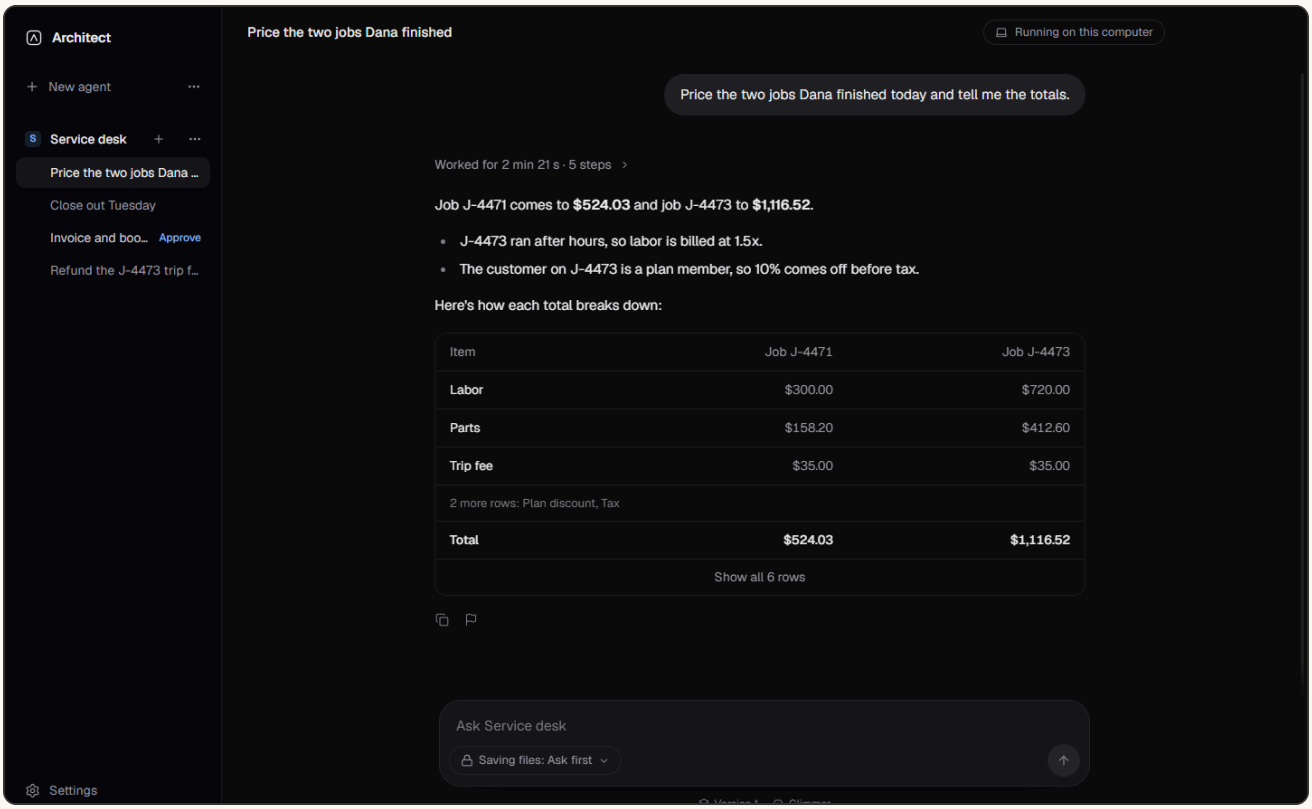



Figure 12

### Files and folders

The agent sees only the folders you gave it, and only the files its job names. Press the three dots beside the agent's name, then **Permissions**. Each folder shows the place on your computer it points to, with a button to choose another. Changing one runs this computer's checks again.

### Approving changes

**Saving files** sits under the message box. A new agent starts on **Ask first**. When the agent wants to change a file, a card shows the file and the rows or lines it would write, and the reply waits: **Waiting for your approval**. In the sidebar, a chat that waits shows **Approve**. See **Figure 13**.

| Button | What it does |
|---|---|
| **Allow once** | Saves this change. |
| **Deny** | Does not save it. |
| **Allow saving from now on** | Saves this change and later ones without asking. It sets **Saving files** to **Allow**. Each change still shows in the chat. |

Set **Saving files** to **Never** if the agent should only read.

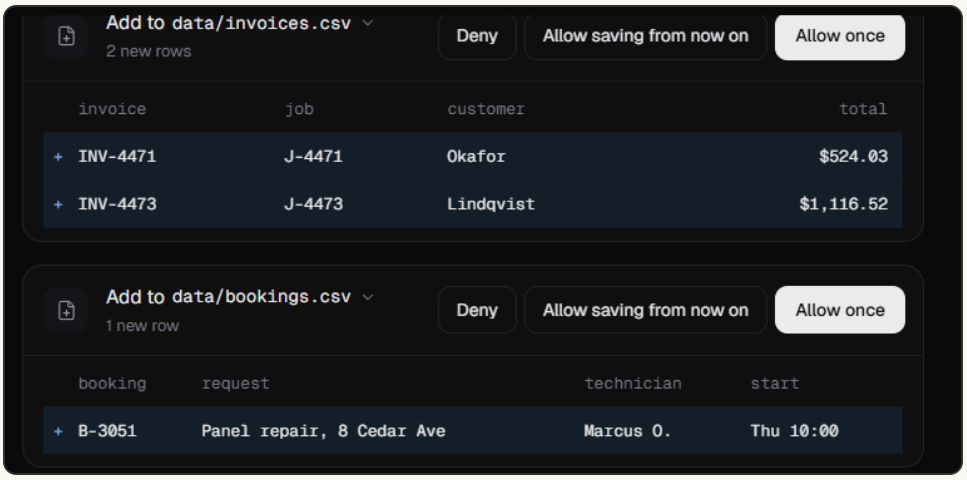



Figure 13

### Working offline

The pill at the top says **Running on this computer**. The agent talks only to the model on your computer. You can turn off the internet and keep using it. The first reply may be slow while the model loads. If another job is using the graphics card, the step says **Waiting: your computer is busy with another job.** and goes on when the card is free.

### What it refuses

An agent does its job and nothing else. Ask for something outside it and it says so, with the reason, instead of guessing. What it leaves to you is listed on the Results screen under **What it won't do**.

## If something goes wrong

| If you see | Do this |
|---|---|
| **A slow reply** | Glimmer is slow on a computer without much graphics memory. Architect plans for about 2.5 minutes a task. A shorter request helps. |
| **Not saved on a change you allowed** | The card says why. Press **Try again**, or **Don't save**. |
| **Not applied** | You stopped the reply before answering, so the change was not made. |
| **This task needs more room than the model's context allows** | The request and what the agent had to read didn't fit. Ask for less at once. |
| **Can't run on this computer** | Press the three dots beside the agent's name, then **General**. **Works on this computer** says what failed and offers a fix. |

# 10 Make it better

When the agent gets something wrong, say so. You choose what happens next, and the version in use never changes by itself.

### Teach this

1. Under the wrong reply, press the flag, **Flag for teaching**. A panel called **Teach this** opens. See **Figure 14**.
2. In **What was wrong**, say what it got wrong. Two more boxes are optional: **What it should have done** and **What it should have used**, where you pick files or paste rows.
3. Read the line at the bottom. Only this exchange, its steps and your notes go to Spark, for up to a small cost. Nothing goes until you press **Share with Spark**.
4. Spark proposes a lesson. It says what kind it is, what change it would make, whether practice is needed, and what reading it cost. See **Figure 15**.

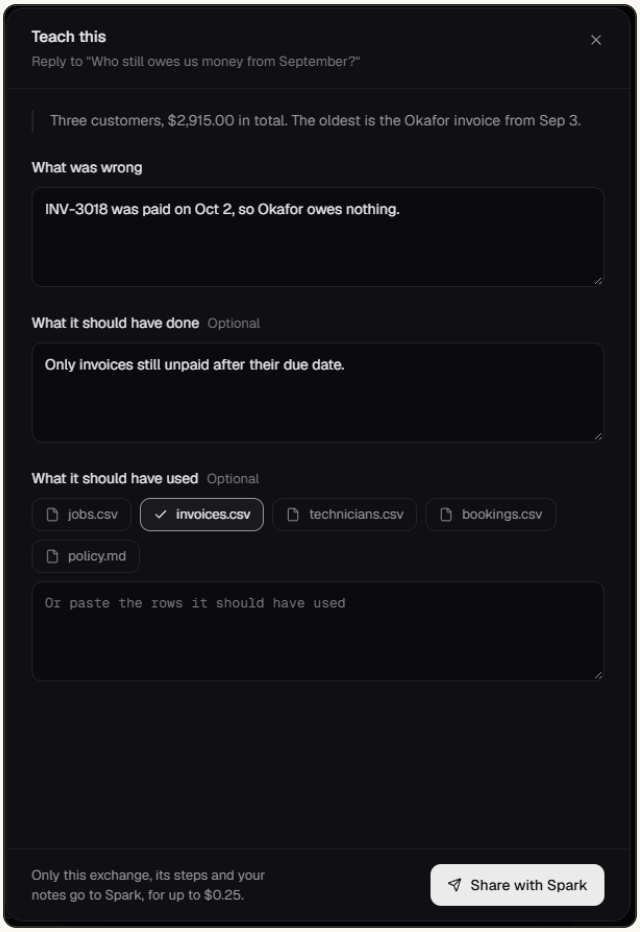



Figure 14

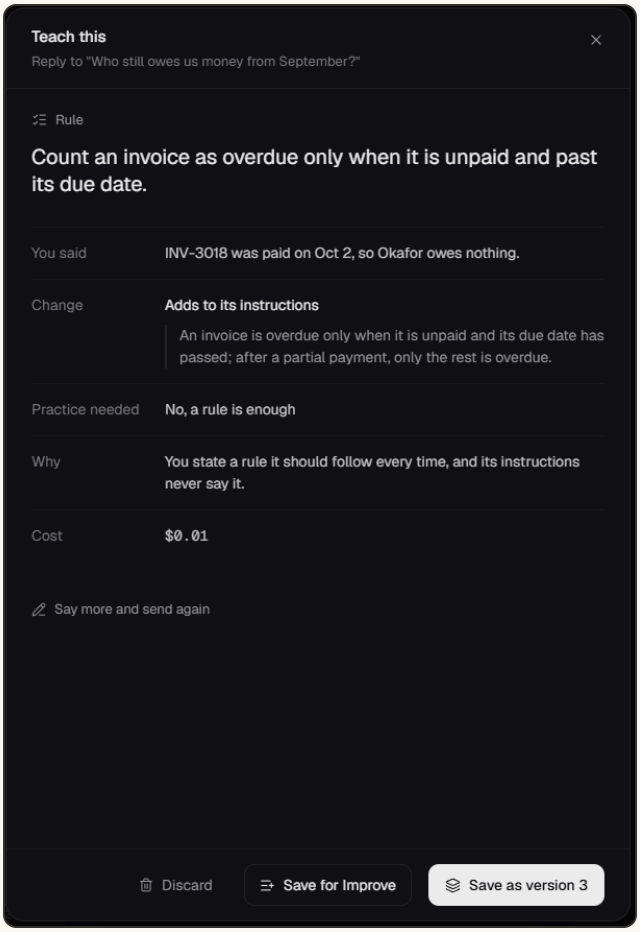



Figure 15

## Kinds of lesson

| Kind | What happens |
|---|---|
| **Rule** | You state something it should do every time. It is added to the agent's instructions. No practice is needed. |
| **Missing fact** | It lacked a fact or a file. This is added to its instructions too. |
| **Needs practice** | It keeps making the same kind of mistake. Improve practices tasks like it. |
| **Unclear request** | Nothing changes until you say which you meant. |
| **Can't be checked** | No change. |

For a rule or a missing fact, **Save as version** and its number adds the lesson to the agent as a new version at once. That version isn't used until you choose it, and it hasn't taken a final test. **Save for Improve** keeps the lesson with your other flagged answers. **Discard** throws it away. **Say more and send again** lets you add detail.

### Improve

Improve teaches your saved lessons together. Press the three dots beside the agent's name, then **Teaching**. Under **Lessons**, **Flagged answers** lists what you flagged, and **Improve** starts the run.

1. Press **Improve**. A box asks **Improve** and the agent's name, and says what will happen: it teaches your lessons to a copy of the current version, saves a new version only if it scores better on the check tasks, and stops at the caps it shows. The current version keeps answering meanwhile.
2. Press **Start improving**. The Train screen opens, with the steps **Lessons**, **Improve** and **Results**. You can pause or stop it.
3. When it ends, press **See what changed**. The screen shows the score before and after on the check tasks. A new version is marked **Not in use yet**, and it hasn't taken a final test.

### Versions

**Versions**, under **Teaching**, lists every version of the agent, newest first. Each is **In use**, **New, not in use yet**, **Earlier version** or **Set aside**, and shows its final test result, **No final test yet**, or how far it fell short of its target. See **Figure 16**.

- To use a new version, press **Use this version**. A box says what changes: new replies use that version's setup and trained weights, and the version you leave stays.
- To go back, press **Go back to this version** on an earlier one. It restores that version's setup and weights together.
- A version that came from another computer never writes without asking. Saving files goes back to **Ask first** when you use it.

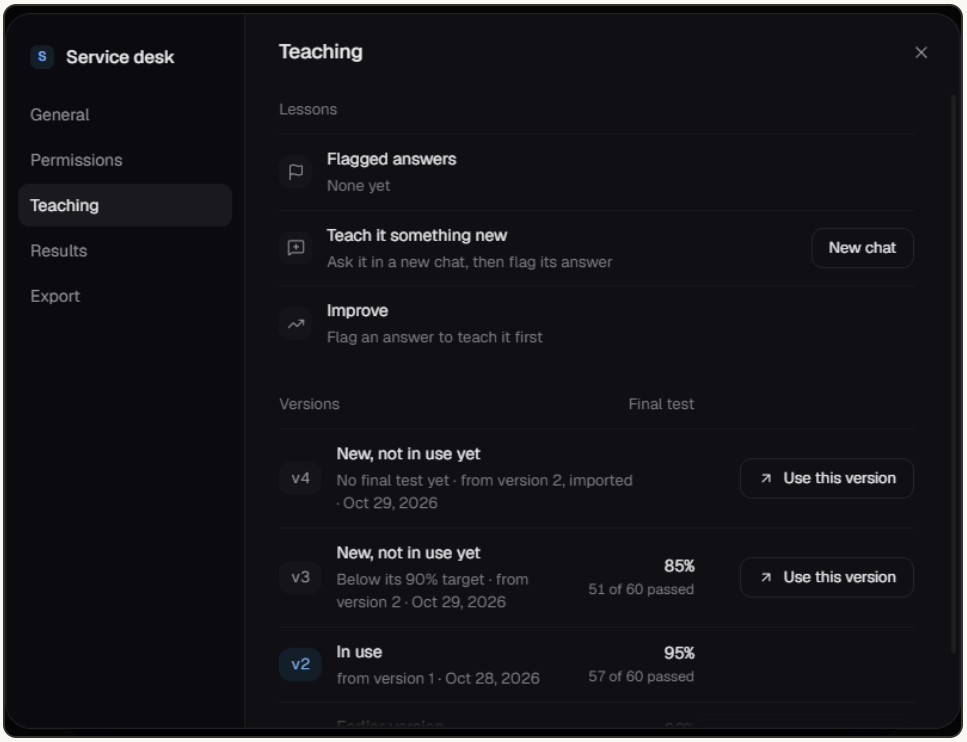



Figure 16

## If something goes wrong

| If you see | Do this |
|---|---|
| **Can't be taught on this computer** | The row says why. An agent that came from another computer may need its model or runtime set up first. |
| **Not shared** | The exchange matches a question from the final test, which Spark must never see. Flag a different answer. |
| **Too long to review** | Reading it would cost more than the limit. Shorten your note, or flag a shorter answer. |
| **No new version** | No change scored better on the check tasks. The current version stays, and the lessons go back in the queue. |

# 11 Share it

Export an agent as one file, publish it to Hugging Face, or install one that someone sent you.

### Export a package file

1. Press the three dots beside the agent's name, then **Export**. Choose **Package file**. See **Figure 17**.
2. Read **In the package**: the setup, the model's name, the job and the test results, plus trained weights if it has them. The file is named like `service-desk-v2.architect.zip`.
3. Read **Stays on this computer**: your folder choices and model settings, your API keys, and the attempts and notes from training. Chats stay too.
4. A folder's files go only if you switch them on, with the switch beside **Files in data/**.
5. Read **Scan**. It looks for lines that look like a key or a private path. If it finds some, read each, then tick **Export anyway**.
6. Press **Export** and choose a folder. **Export one part** saves only the setup, the trained weights or the results.

> The scan is a best effort. Trained weights may keep traces of the data they learned from, and a setup written from your tasks may quote them. Leaving files out doesn't make an agent anonymous.

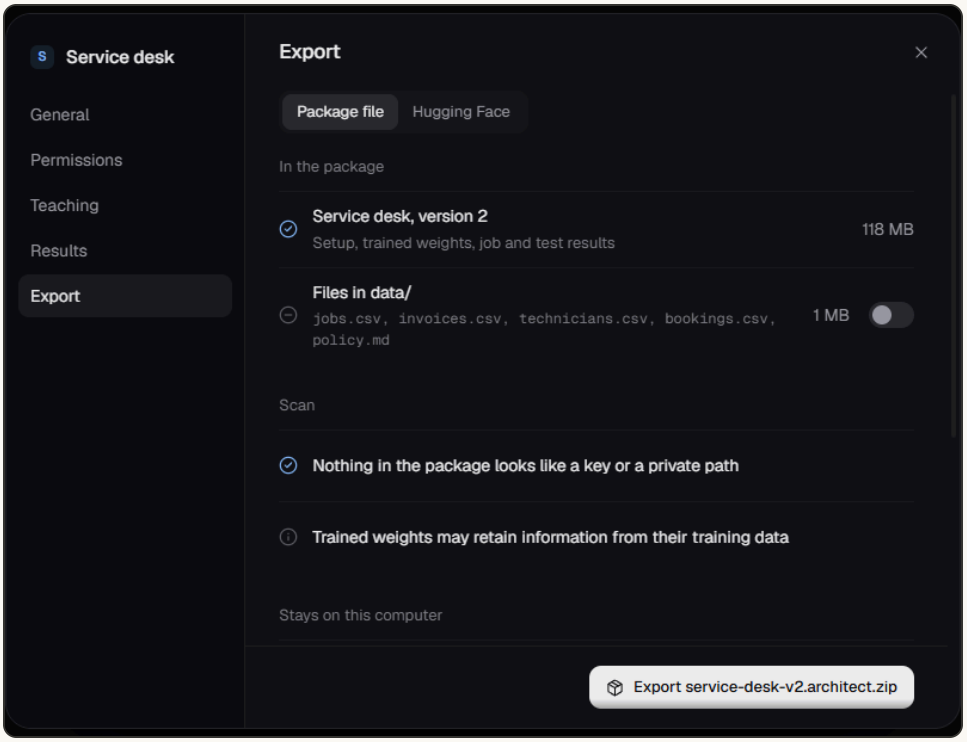



Figure 17

### Publish to Hugging Face

1. In the same tab, choose **Hugging Face**. See **Figure 18**.
2. Type the repository as **owner/name**. Choose **Private**, which is the default and lets only accounts you allow see it, or **Public**, which lets anyone see and download it.
3. Check **Token**. It comes from Settings (Models, then Hugging Face), from `HF_TOKEN` on your computer, or from `hf auth login`. It goes only to Hugging Face's own service, never into the package.
4. Read **Files**, press **Preview** to read the model card, and read **Scan**.
5. Press **Publish version** and its number. The panel shows progress, then the repository and its commit, with the command that installs exactly that commit.

An existing repository keeps its visibility: a private one stays private. If a publication fails, it says **Didn't finish**. Press **Finish publishing** to complete the same one. The agent still runs with Architect on the computer that installs it, not on Hugging Face.

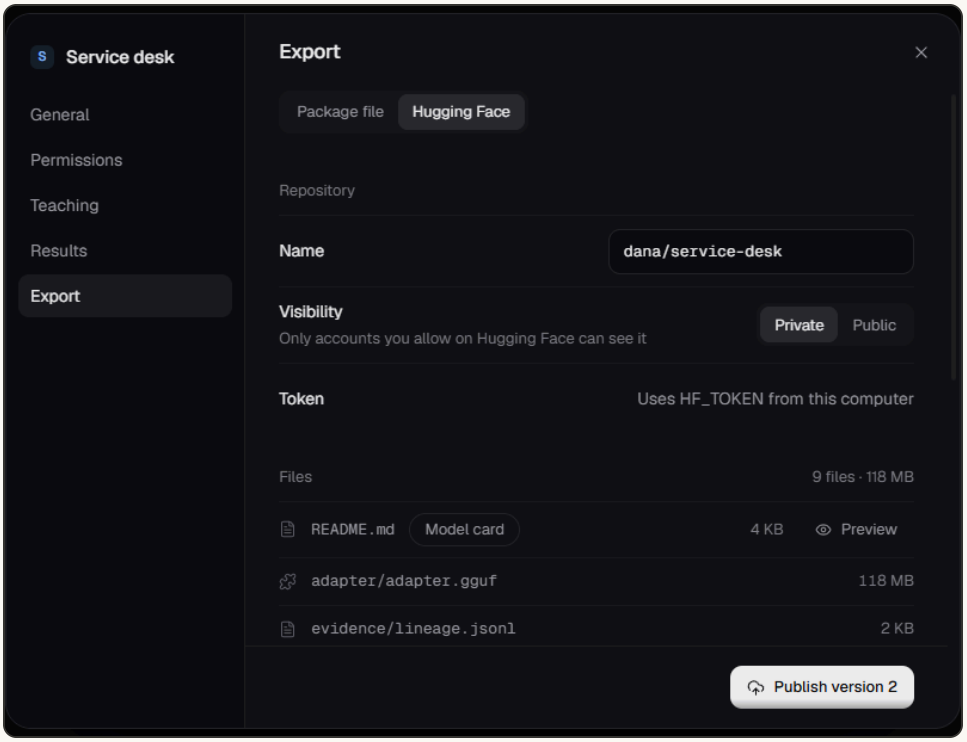



Figure 18

## Import

1. Press the three dots beside **New agent** in the sidebar. Choose **Import agent** for a file or folder, or **Install from Hugging Face** and type **owner/name**. A commit is optional.
2. **Package.** Architect checks the package without running any of it and says **Safe to open**. It shows whether your computer can run it and has the model, and how it did for its author, labelled **Tested by its author**. See **Figure 19**.
3. **Folders. Where are its files?** Choose a folder on your computer for each folder the agent works in.
4. **Permissions. What may it do?** For each folder choose **Not allowed**, **Read** or **Read and change**, never more than the package asks for. **Internet access** reads **Never, by design**.
5. **Install.** Architect installs it and runs its checks. When they pass, press **Start a chat**.

A package asks for access, and nothing is granted unless you grant it. The author's score is not a test on your files. The checks on your computer show that the agent runs here. They are not a new final test. A package that is a new version of an agent you already have joins as a version and waits until you choose to use it.

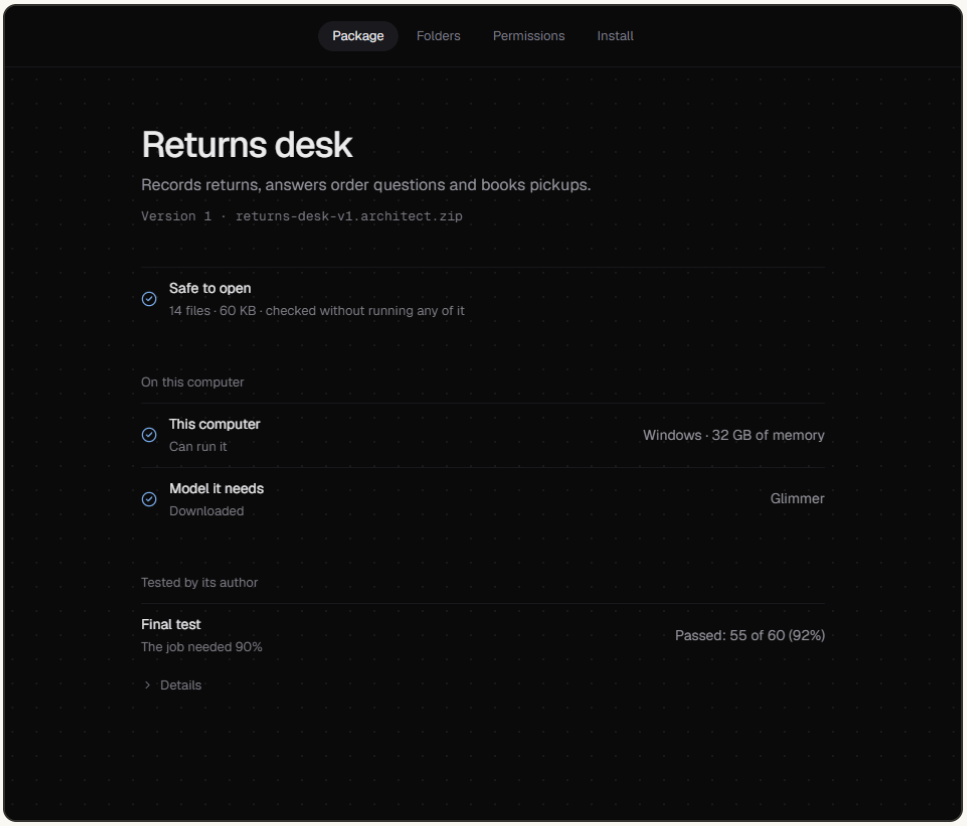



Figure 19

## If something goes wrong

| If you see | Do this |
|---|---|
| **Couldn't check this package, or a list of problems** | Choose another file. If the message says the package was edited after it was made, ask whoever sent it for a new copy. |
| **Add a Hugging Face token in Settings** | Publishing needs a token. Create one at huggingface.co, then add it in Settings, under Models. |
| **Flagged lines need your OK** | The scan found lines that look like keys or private paths. Read them. If they are fine, tick **Export anyway** or **Publish anyway**. If not, take them out and export again. |
| **Didn't finish** | Publish again to the same repository. It finishes the same publication without resending what Hugging Face already has. |

# 12 The command line

The `architect` command lists, installs, runs, checks, exports and publishes agents from a terminal, without opening the app.

## What it is

It works on the same agents as the app, in the same folder, with the same rules. An agent sees only the folders you gave it, asks before it changes a file unless you allow it, and runs only on a model on your computer. The teacher is never contacted. Only `publish`, and `install` from Hugging Face, go online, and only to Hugging Face.

## Getting it

The command comes with the installer.

**Windows** `architect.cmd`, beside `Architect.exe` in the folder the app installed to (`%LOCALAPPDATA%\Programs\Architect` by default). Add that folder to `PATH` to type `architect` anywhere.

**macOS** `Architect.app/Contents/Resources/architect`. Link it into a folder on your `PATH`.

```
ln -s /Applications/Architect.app/Contents/Resources/architect \
  /usr/local/bin/architect
```

## Commands

| Command | What it does |
|---|---|
| `architect list` | Lists every agent on this computer, with its name, id, active version and whether it can run here. |
| `architect install` | Installs a package, or a version published to Hugging Face. Everything is checked before any of its code loads. |
| `architect run` | Talks to an agent, or runs one task from a file. |
| `architect inspect` | Shows an agent's versions, model, folders, permissions, the author's results and this computer's checks. Given a package file, it checks the package without installing it. |
| `architect verify` | Runs the active version's installation checks again. It is not an evaluation. |

| Command | What it does |
| --- | --- |
| `architect export` | Writes a version as one package file, after the same review the app shows. |
| `architect publish` | Publishes a version to a Hugging Face repository, private unless you add `--public`. `--dry-run` shows what would go and sends nothing. |

An agent is named by the name `architect list` shows, or by its id. Add `--json` to `list`, `install`, `inspect`, `verify`, `export` and `publish` for output a script can read.

```
architect install ./desk.architect.zip --bind data=./files
architect install hf://jake/service-desk --revision 3f1c9e2a
architect export service-desk --output ./desk.architect.zip
architect publish service-desk --repo jake/service-desk --private
```

A package asks for access and nothing is granted that the command doesn't name. `--bind` points one of the package's folders at a folder here, and `--grant` gives it `read`, `read_write` or `none`. `--writes` takes `ask` (the default), `allow` or `never`.

### Talking to an agent

`architect run service-desk` starts a conversation. Type one request per line. The answer streams to the terminal and the steps go to stderr. A change that needs approval is asked about in the terminal. Ctrl+C stops a reply, and Ctrl+D ends the conversation.

```
$ architect run service-desk
> Who owes us money?
  read data/invoices.csv (3 rows)
Ruiz, Ben owes $45.50 on INV-2, due November 15.
```

### Running one task from a script

Give `run` a request as a JSON file, and it writes the result as JSON. Results go to stdout and everything else to stderr, so a script can read stdout.

```
architect run service-desk --input request.json --output result.json
--non-interactive
```

```
{
  "format": "architect.run-request/1",
  "task": "Who owes us money, and how much?",
  "inputs": ["data/invoices.csv"],
  "version": 2
}
```

`task` is what to do, in the words you would use in the app. `inputs` and `version` are optional. `--non-interactive` means nobody can be asked, so a change that needs approval ends the run with the error `approval_required`, and nothing is written. The result has a `status`, an `exit_code`, the `answer`, the `steps` taken, and an `error` with a `code` and a `hint` when something went wrong.

## Exit codes

| Code | Outcome | When |
|---|---|---|
| **0** | succeeded | It did what was asked. |
| **1** | runtime_error | Something failed while running: the model runtime isn't there, a time limit ran out, or Hugging Face couldn't be reached. |
| **2** | invalid_input | The command line or the request is malformed, or names something that isn't there. |
| **3** | not_permitted | A requirement or permission isn't met: a change that needs approval, a folder that isn't bound or granted, a model that isn't on this computer, or a missing token. |
| **4** | task_failed | The agent ran and didn't do the task. It refused it, or stopped without an answer. |
| **130** | cancelled | Ctrl+C or a stop signal ended it. |

## If something goes wrong

| If you see | Do this |
|---|---|
| **Exit 3 after** `install` | A folder the package needs is not bound or granted. The checks say which. Run again with `--bind` and `--grant`. |
| **Exit 1, and the model isn't answering** | Start Ollama. `architect verify` checks the agent again. |
| **The command waits, then exits 1** | The app is using the graphics card. The command waits up to 5 minutes, then says what holds it. |
| **Exit 3 after** `export` **or** `publish`**, with a list** | The scan found lines that look like a key or a private path. Check them. If they are fine, run again with `--accept-findings` and the id it printed. |

# 13 What leaves your computer

Most of Architect stays on your computer. This chapter says exactly what doesn't, and what never does.

## What can leave

| When | What goes | To whom |
|---|---|---|
| **You describe a job** | Your description and the files you attach. A text file goes whole. For a table, Spark sees its columns, examples of what is in them, and the first five rows. | Your teacher. Nothing leaves if the teacher runs on your computer. |
| **It trains** | Practice attempts and samples of your files, so Spark can diagnose failures. For the check tasks, only the score. | Your teacher. |
| **Practice online, if you choose it** | Tasks and files. | The online host you chose. |
| **You use Teach this** | After you confirm, the one exchange you flag, its steps and your notes, for a capped cost. | Your teacher. |
| **You download a model** | The download. | Ollama, or Hugging Face. |
| **You publish or install from Hugging Face** | The package you choose. Your token goes only to Hugging Face's own service. | Hugging Face. |

Each teacher has its own data terms. For Spark, the app says: Meta doesn't train on what you send. Read your provider's terms if you use another teacher. Settings, under Privacy, gives the short version.

## What never leaves

- **The final test.** Spark never sees its requests, its answers or its scores before the agent graduates. Architect checks everything it sends to Spark against the final test, and refuses to send a match. Nothing about the final test goes back into training.
- **Your chats.** Spark never sees them. The one exception is Teach this, and only after you press **Share with Spark**.
- **Your agent's work.** A finished agent talks only to the model on your computer. Architect refuses any other connection, and an agent never calls Spark.

- **Usage data.** Architect collects none.
- **Your keys.** They are stored encrypted on your computer. They never go into a package.

### What Spark can and can't change

Spark can change an agent's harness: its instructions, what it reads and when, and small tools. It can't change how answers are graded, the task sets, the final test, or what the agent may access.

The small tools Spark writes run in a sandbox. They have no network and no clock, and they can read only the files the job names, without changing them. A change to your files needs your approval unless you set **Saving files** to **Allow**.

Rounds, Spark's dollars and time are hard caps. A call that would go over one does not start.

### If you want to share less

- Use a teacher that runs on your computer, in Ollama or LM Studio. Then nothing leaves.
- Attach only the files the job needs. For a policy, attach only the section it needs.
- Choose **Train on this computer** on the Define screen instead of **Train online**.
- Under each choice, the app says what leaves. Read it before you press the button.

# 14 When something goes wrong

The common problems, by where you meet them. Each chapter also lists the problems of its own steps.

### Setting up

| If you see | Do this |
|---|---|
| **Ollama isn't running** | Open Ollama, or install it from ollama.com. Then press **Check again**. |
| **Windows or macOS warns about the app** | The app isn't signed yet. On Windows, choose **More info**, then **Run anyway**. On a Mac, right-click it and choose **Open**. |
| **Part runs on the processor, so it is slower** | The build is too large for your graphics memory. Choose the smaller build, or accept the slower speed. |
| **The teacher's key test fails** | Check that you pasted the whole key. Add or replace keys in Settings, under Models. |

### Describing and defining

| | |
|---|---|
| **Your files hold only a few rows** | Attach more records, or at least 30 example tasks in a `tasks.jsonl` file. |
| **A file can't be read yet** | Architect reads CSV, text and Markdown. Export a spreadsheet, PDF or Word file as one of those. |
| **Start training is off** | Read the line beside it. It says **Confirm the rule first**, **Describe the job again first**, or what is missing. |
| **Changed since you confirmed it** | Your records changed after you confirmed the rule. Press **Describe the job again**. |
| **Not even one round of teaching fits** | Allow more time, or train online (chapter 6). |

## Training

| | |
|---|---|
| **Training is paused** | Read what is missing, fix it with the button beside it, and press **Resume**. |
| **Spark has a question for you** | Answer it in the round panel. Training waits for you. |
| **Training stopped on an error** | Read the message, then press **Back to the job**. |
| **Spark's budget ran out** | Raise the cap in **Advanced settings** and train again, or improve the agent later. |
| **Not ready on the results** | You can still use the agent. Teach it more, or train again with more files. |

## Using an agent

| | |
|---|---|
| **Waiting: your computer is busy with another job.** | Another run or check is using the graphics card. The reply goes on when it is free. |
| **Not saved** | Press **Try again** on the card, or **Don't save**. |
| **This task needs more room than the model's context allows** | Ask for less at once. |
| **Can't run on this computer** | Open the agent's settings, then **General**. Use the fix button beside the failed check. |

## Sharing

| | |
|---|---|
| **Couldn't check this package** | Choose another file, or ask the sender for a new copy. |
| **Lines that look like a key or a private path** | Read each one. Take out anything private, then export or publish again. |
| **Didn't finish on a publication** | Publish again to the same repository. It finishes the same publication. |
| **Add a Hugging Face token in Settings** | Add a token in Settings, under Models. |

# 15 Glossary

The words this guide and the app use, in the order you will meet them.

| | |
|---|---|
| **Agent** | What Architect makes for a job: Glimmer with its harness. It runs offline on your computer. |
| **Spark** | The teacher. A large model, Muse Spark by default, reached over the internet with a key. |
| **Glimmer** | The student. A small model, Muse Glimmer, that runs on your computer through Ollama. |
| **Ollama** | A free app that runs models on your computer. Glimmer runs through it. |
| **Job** | What you describe: the tasks, the files they use, the limits and the target. |
| **Harness** | The plain folder of files that tells Glimmer how to do the job: instructions, notes it reads, and small tools. Spark edits it. |
| **Rule** | A policy with prices or rounding that Spark turned into exact steps. You confirm it before anything trains. |
| **Round** | One cycle of training: practice, diagnose, change, score. |
| **Practice tasks** | Tasks Glimmer works through so Spark can see where it goes wrong. |
| **Check tasks** | Tasks Spark never sees. Spark gets only their score, and the score decides which changes stay. |
| **Final test** | Tasks put aside before training and opened once, at the end, with Spark disconnected. |
| **Kept, undone** | A change is kept if it gains enough on the check tasks and breaks no rule. Otherwise it is undone. |
| **Graduated** | Passed the final test on this computer. The agent is marked **Ready**. |
| **Version** | A saved state of an agent. A new version waits until you choose to use it, and an earlier one can be restored. |

| | |
|---|---|
| **Teach this** | The flag under a reply. It turns a wrong answer into a lesson. |
| **Improve** | A training run that teaches the saved lessons and makes a new version if it scores better. |
| **Weights** | The numbers inside a model. Retraining them is an option for Spark, and it needs a Hugging Face key and a training budget. |
| **Package** | An agent exported as one `.architect.zip` file. |
| **Hugging Face** | A site where models are shared. Architect can publish an agent to it and install one from it. |